

Task 2 — Customer 360 AI Assistant

Growth Squad Technical Assessment | Volopay | July 2026

Table of Contents

- 1. [Problem Statement](#1-problem-statement)
- 2. [Business Challenge](#2-business-challenge)
- 3. [Solution Overview](#3-solution-overview)
- 4. [Architecture](#4-architecture)
- 5. [Technology Stack](#5-technology-stack)
- 6. [Project Structure](#6-project-structure)
- 7. [Data Schema](#7-data-schema)
- 8. [Health Score Logic](#8-health-score-logic)
- 9. [AI Insights Engine](#9-ai-insights-engine)
- 10. [LLM Integration Hooks](#10-llm-integration-hooks)
- 11. [Application UI Walkthrough](#11-application-ui-walkthrough)
- 12. [Sample Input & Output](#12-sample-input-output)
- 13. [Running Locally](#13-running-locally)
- 14. [Deployment Guide](#14-deployment-guide)
- 15. [Future Roadmap](#15-future-roadmap)

1. Problem Statement

Customer Success teams at modern SaaS companies operate across multiple disconnected data systems — CRM platforms, support ticketing tools, email inboxes, and product analytics dashboards. The result is fragmented visibility: a CSM may know a customer's ARR but not their open critical ticket; a support agent may know about an escalation but not that renewal is 14 days away.

This fragmentation leads to **reactive account management**, **missed upsell windows**, and **avoidable churn**.

2. Business Challenge

Challenge	Impact
No unified account view	CSMs make decisions without full context
Manual health assessment	Inconsistent, subjective scoring
Reactive escalation	Critical issues surface too late
No next-action guidance	Valuable accounts receive generic treatment
LLM integration complexity	Teams unsure how to safely add AI to CS workflows

3. Solution Overview

Customer 360 AI Assistant is a production-ready Streamlit application that aggregates four data sources into a single intelligent account view and applies a deterministic rule engine to generate:

- A weighted **Health Score** (0–100) with transparent signal breakdown
- A prose **Customer Summary** ready for internal briefing
- A prioritized **Business Risks** list
- An **Opportunities** list for upsell and expansion
- A single, context-aware **Next Best Action** with owner and urgency

The engine is **fully functional without an API key**, and has clearly marked # LLM_H00K integration points for OpenAI GPT-4 or Google Gemini activation.

4. Architecture

```

flowchart TD
    subgraph DATA ["■ Data Sources"]
        A["crm.csv\n(CRM Records)"]
        B["support.csv\n(Support Tickets)"]
        C["emails.csv\n(Email History)"]
        D["usage.csv\n(Product Usage)"]
    end

    subgraph UTILS ["■ utils.py – Intelligence Engine"]
        E["load_all_data()\nData Loaders"]
        F["get_customer_profile()\nProfile Builder"]
        G["compute_health_score()\nWeighted Scoring Engine"]
        H["generate_customer_summary()\nProse Generator"]
        I["generate_business_risks()\nRisk Detector"]
        J["generate_business_opportunities()\nOpportunity Finder"]
        K["generate_next_best_action()\nPriority Action Engine"]
        L["LLM_H00K: call_openai()\ncall_gemini()"]
    end

    subgraph APP ["■ app.py – Streamlit UI"]
        M["Sidebar\nCustomer Selector + AI Mode"]
        N["CRM Overview\nMetric Cards"]
        O["Support History\nStyled Ticket Table"]
        P["Email Summary\nBody + AI Summary"]
        Q["Usage Metrics\nProduct Engagement"]
        R["AI Insights Tabs\nSummary | Risks | Opps | NBA | Health"]
    end

    subgraph DEPLOY ["■ Deployment"]
        S["Streamlit Community Cloud"]
        T["Render Web Service"]
    end

    A --> E
    B --> E
    C --> E
    D --> E
    E --> F
    F --> G
    F --> H
    F --> I
    F --> J
    G & H & I & J --> K
    H & I & J & K -.-> L
    L -.-> K
    K --> APP
    APP --> DEPLOY
  
```

Data Flow (Step-by-Step)

1. User selects an account from the sidebar dropdown
2. app.py retrieves @st.cache_data DataFrames via load_all_data()
3. get_customer_profile() merges all four sources by customer_id
4. compute_health_score() applies weighted rule engine → score + breakdown dict
5. Four generators produce: Summary, Risks, Opportunities, Next Best Action
6. UI renders all sections with color-coded metrics and styled components
7. (Optional) If API key is set, LLM hooks replace deterministic generators

5. Technology Stack

Layer	Technology	Version
UI Framework	Streamlit	≥ 1.32.0
Data Processing	Pandas	≥ 2.1.0
File I/O	openpyxl	≥ 3.1.2
Python	CPython	3.10+
Styling	Custom CSS via `st.markdown`	—
LLM (optional)	OpenAI GPT-4o / Google Gemini 1.5 Pro	—
Deployment	Streamlit Community Cloud / Render	—

6. Project Structure

```
Task2/
├── app.py # Main Streamlit application (UI + rendering logic)
├── utils.py # Data loaders, health engine, AI insight generators
├── requirements.txt # Python dependencies
├── packages.txt # System-level packages for cloud deployment
├── README.md # Setup and deployment guide
├── data/
│   ├── crm.csv # CRM records (15 accounts)
│   ├── support.csv # Support tickets (20 tickets)
│   ├── emails.csv # Latest email per account
│   └── usage.csv # Monthly product usage metrics
```

7. Data Schema

crm.csv — 15 Records

Simulates a B2B SaaS CRM. Companies are drawn from realistic industries (logistics, fintech, pharma, manufacturing). Plans range from Starter (\$12K ARR) to Enterprise (\$300K ARR). Renewal dates are within the next 12 months to create urgency scenarios.

Column	Type	Description
`customer_id`	string	Primary key (C001–C015)
`company`	string	Company name
`owner`	string	Customer Success Manager
`plan`	string	Starter / Growth / Enterprise
`ARR`	integer	Annual Recurring Revenue (USD)
`renewal_date`	date	Contract renewal date

support.csv — 20 Records

Covers all 15 customers with varying ticket counts (0–2 per customer). Includes Critical, High, Medium, and Low priorities. Sentiment labels simulate AI classification output.

Column	Type	Description
`customer_id`	string	Foreign key to crm.csv
`ticket_id`	string	Unique ticket identifier
`ticket`	string	Ticket description
`priority`	string	Critical / High / Medium / Low
`status`	string	Open / Resolved / In Progress
`sentiment`	string	Negative / Neutral / Positive
`created_date`	date	Ticket creation date

emails.csv — 15 Records

One email record per customer. Email bodies are realistic business communication including escalations, renewal inquiries, and positive feedback.

Column	Type	Description
`customer_id`	string	Foreign key to crm.csv
`email_date`	date	Email date
`sender`	string	Sender email address
`subject`	string	Email subject line
`latest_email`	string	Full email body
`email_summary`	string	AI-generated summary (simulated)

usage.csv — 15 Records

Monthly product engagement data per customer.

Column	Type	Description
`customer_id`	string	Foreign key to crm.csv
`month`	string	Reporting month (YYYY-MM)
`invoices_processed`	integer	Invoices processed that month
`payments_made`	integer	Payments made that month
`active_users`	integer	Number of active users
`login_frequency`	string	Daily / Weekly / Monthly
`api_calls`	integer	Total API calls
`feature_adoption_score`	integer	Composite score (0–100)

8. Health Score Logic

The health score is a **weighted penalty/bonus system** starting from a base of 100 and applying deductions based on negative signals. Final score is clamped between 0 and 100.

Signal	Condition	Points
Critical tickets	Each critical open ticket	−20 pts (max −40)
Open ticket count	Each ticket beyond 2	−5 pts (max −20)
Negative sentiment	Each negative ticket beyond 1	−5 pts (max −15)
Renewal urgency	Renewal < 30 days	−15 pts
Renewal warning	Renewal 30–60 days	−8 pts
Low feature adoption	Adoption score < 40	−10 pts
Low login frequency	Monthly logins only	−10 pts
Churn language in email	Detected churn keywords	−10 pts
High adoption bonus	Adoption score > 80	+5 pts

Health Score Labels

Score Range	Label	Badge Color
75–100	■ Healthy	Green
50–74	■ Needs Attention	Yellow
25–49	■ At Risk	Orange
0–24	■ Critical	Red

9. AI Insights Engine

All five insight generators live in `utils.py` and operate deterministically — no external API required.

`generate_customer_summary(profile)`

Constructs a 3–4 sentence executive briefing covering: plan + ARR, renewal urgency, critical ticket status, and overall health. Output is ready for internal team communication.

`generate_business_risks(profile)`

Returns a ranked list of risks. Detects:

- Critical ticket escalation risk
- Renewal urgency (< 30 days, < 60 days)
- Churn language in recent emails
- Low product adoption
- Infrequent login patterns

`generate_business_opportunities(profile)`

Returns actionable opportunities based on signals:

- High adoption → Upsell to next tier
- High API usage → Developer seat expansion
- Positive sentiment → Reference / case study candidate
- Enterprise plan → Executive sponsorship program

generate_next_best_action(profile)

Returns **one** highest-priority action with:

- Urgency level: ■ Immediate / ■ This week / ■ This month
- Action description
- Owner (assigned CSM)
- Rationale

compute_health_score(profile)

Returns (score: int, breakdown: dict) where the breakdown exposes every signal and its point impact for transparent UI display.

10. LLM Integration Hooks

The app ships with clearly marked # LLM_HOOK comments at every AI generation point in `utils.py`. To activate LLM-powered outputs:

Step 1 — Choose your provider:

```
# In utils.py, locate: # LLM_HOOK
# Uncomment one of the following blocks:

# --- OpenAI GPT-4 ---
import openai
client = openai.OpenAI(api_key=st.session_state.get("api_key"))

# --- Google Gemini ---
import google.generativeai as genai
genai.configure(api_key=st.session_state.get("api_key"))
```

Step 2 — Prompt template used:

```
You are a Customer Success Manager AI assistant. Based on the following
customer profile data, write a concise 3-4 sentence executive summary
that can be used for internal team briefings.

Customer: {company}
Plan: {plan} | ARR: ${arr:,}
Days to Renewal: {days_to_renewal}
Open Tickets: {open_tickets} | Critical: {critical_tickets}
Health Score: {health_score}/100 ({health_label})
Email Summary: {email_summary}
Feature Adoption Score: {feature_adoption_score}

Focus on:
- Current account status and risk level
- Most urgent action required
- Revenue implications
Keep the tone professional and data-driven.
```

Step 3 — Enter API key in app sidebar (labeled "Optional: OpenAI / Gemini API Key")

No code changes needed. The sidebar key input feeds directly into the LLM_HOOK functions.

11. Application UI Walkthrough

Sidebar

- **Customer Selector** — Dropdown showing all 15 accounts (format: Company Name (ID))
- **AI Mode Toggle** — Switch between Rule-Based and LLM-powered insights
- **API Key Input** — Secure password field for OpenAI / Gemini key

Main Panel — 5 Data Sections

Section	What It Shows
CRM Overview	4 metric cards: Plan, ARR, Renewal Date, Days to Renewal
Support History	Color-coded table with priority badges (Critical=red, High=orange) and sentiment labels
Email Summary	Full email body in an expander + AI-generated summary highlighted box
Usage Metrics	4 metric cards: Invoices, Payments, Active Users, API Calls + Adoption score
AI Insights	5 tabbed panels (Summary · Business Risks · Opportunities · Next Best Action · Health Score)

AI Insights Tabs

Tab	Output
Summary	3–4 sentence executive briefing
Business Risks	Bulleted risk list ranked by severity
Opportunities	Bulleted opportunities for upsell / expansion
Next Best Action	Single action card with urgency, owner, rationale
Health Score	Gauge metric + signal-by-signal breakdown table

12. Sample Input & Output

Input: Crimson Media Group (C009)

CRM: Enterprise | ARR: \$300,000 | Renewal: 31 Jan 2025 | Owner: Priya Sharma

Support: 2 tickets – both Critical, both Open, both Negative sentiment

Email: "We have two open critical issues and our team is running out of patience. We process \$2M+ in payments monthly..."

Usage: 1,243 invoices | 23,450 API calls | Adoption: 95/100

Output

Health Score: 20 / 100 — ■ Critical

Customer Summary:

Crimson Media Group is on the Enterprise plan with an ARR of \$300,000, managed by Priya Sharma. Their renewal is in 42 days, making this an urgent account to prioritize. There are 2 critical support tickets currently open, which presents an escalation risk and requires immediate CSM attention. Overall account health is rated Critical (20/100).

Top Risk:

■ Critical Support Escalation — 2 critical tickets unresolved at \$300K ARR.

Next Best Action:

Escalate 2 critical tickets to Engineering with VP-level visibility.

- **Urgency:** ■ Immediate (< 24 hrs)

13. Running Locally

```
# Step 1 – Navigate to the Task2 directory
cd Volopay_Assessment/Task2

# Step 2 – Install Python dependencies
pip install -r requirements.txt

# Step 3 – Launch the app
streamlit run app.py
```

The app opens at `http://localhost:8501` automatically.

Requirements

```
streamlit>=1.32.0
pandas>=2.1.0
openpyxl>=3.1.2
```

Optional LLM packages (uncomment in `requirements.txt` when activating):

```
'''
```

```
openai>=1.14.0
```

```
google-generativeai>=0.4.0
```

```
'''
```

14. Deployment Guide

Option A — Streamlit Community Cloud (Free)

1. Push the Task2/ directory to a **public GitHub repository**
2. Go to share.streamlit.io and sign in with GitHub
3. Click **New App** → select your repo
4. Set **Branch:** main | **Main file path:** Task2/app.py
5. Click **Deploy**

Add API keys via **Manage App** → **Secrets** (TOML format):

```
```toml
```

```
OPENAI_API_KEY = "sk-..."
```

```
```
```

Option B — Render Web Service

1. Create a new **Web Service** on render.com
2. Connect your GitHub repository
3. Set **Build Command:** `pip install -r requirements.txt`
4. Set **Start Command:** `streamlit run app.py --server.port $PORT --server.address 0.0.0.0`
5. Add environment variables for API keys

Option C — Docker

```
FROM python:3.11-slim
WORKDIR /app
```

```
COPY . .
RUN pip install -r requirements.txt
EXPOSE 8501
CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0"]

docker build -t customer360 .
docker run -p 8501:8501 customer360
```

15. Future Roadmap

| Improvement | Description | Priority |
|----------------------------|--|----------|
| **LLM Integration** | Full OpenAI / Gemini activation for narrative insights | ■ High |
| **Real CRM Sync** | Connect to Salesforce / HubSpot via API | ■ High |
| **Predictive Churn Model** | ML model trained on historical churn patterns | ■ Medium |
| **Email Ingestion** | Auto-parse emails from Gmail / Outlook via OWA | ■ Medium |
| **Slack Alerts** | Push Next Best Action to CSM's Slack channel | ■ Medium |
| **Multi-language Support** | Generate summaries in regional languages | ■ Low |
| **Mobile Optimization** | Responsive layout for field CSM use | ■ Low |
| **A/B Testing** | Test LLM vs rule-based output quality | ■ Low |

[!NOTE]

Live Demo: The app is running locally at `http://localhost:8501`

Source Files: `C:\Users\RELIANCE\.gemini\antigravity\scratch\Volopay_Assessment\Task2\`